

STACK DATA STRUCTURE: TEST BANK (50

QUESTIONS)

SECTION 1: CONCEPTUAL & CORE OPERATIONS

1. A stack is a data structure that follows which principle?
 - a) FIFO (First-In, First-Out)
 - b) LIFO (Last-In, First-Out)
 - c) LILO (Last-In, Last-Out)
 - d) FIGO (First-In, Garbage-Out)
2. Which of the following real-world scenarios is NOT a correct analogy for a stack?
 - a) A stack of plates in a cafeteria.
 - b) The "Undo" feature in a text editor.
 - c) A line of people waiting to buy tickets at a cinema.
 - d) The "Back" button functionality in a web browser.
3. If you push the elements 1, 2, 3, 4, and 5 in that order onto a stack, what will be the order of elements when they are popped?
 - a) 1, 2, 3, 4, 5
 - b) 5, 4, 3, 2, 1
 - c) 1, 5, 2, 4, 3
 - d) 2, 3, 4, 5, 1
4. Consider a stack S. The following operations are performed: push(10), push(20), pop(), push(30), push(40), pop(), peek(). What value will peek() return?
 - a) 10
 - b) 20
 - c) 30
 - d) 40
5. (Tricky) Suppose elements 1, 2, and 3 are pushed onto a stack in that order. Which of the following is NOT a possible sequence of popped elements? (Assume you can interleave push and pop operations.)
 - a) 1, 2, 3
 - b) 3, 2, 1
 - c) 2, 1, 3
 - d) 3, 1, 2
6. What is the primary difference between the pop() and peek() operations?
 - a) pop() returns the top element, while peek() returns the bottom element.
 - b) pop() removes and returns the top element, while peek() returns the top element without removing it.
 - c) peek() removes and returns the top element, while pop() returns the top element without removing it.
 - d) There is no difference; they are aliases for the same operation.
7. Attempting to call pop() or peek() on an empty stack is known as:
 - a) Stack Overflow
 - b) Stack Underflow
 - c) Stack Corruption
 - d) Empty Stack Exception
8. In a static (array-based) implementation of a stack, attempting to call push() on a full stack is known as:
 - a) Stack Overflow
 - b) Stack Underflow
 - c) Stack Resize
 - d) Full Stack Exception
9. After the following sequence of operations, what is the size of the stack? push(A), push(B), pop(), push(C), peek(), push(D), pop(), pop()
 - a) 0
 - b) 1
 - c) 2
 - d) 3
10. Which stack operation is used to check if the stack has any elements?
 - a) size()
 - b) peek()

- c) isFull()
- d) isEmpty()

SECTION 2: ARRAY-BASED IMPLEMENTATION & COMPLEXITY

11. In a typical array-based implementation of a stack, the top variable (tracking the index of the top element) is most commonly initialized to:
- a) 0
 - b) 1
 - c) -1
 - d) MAX_SIZE
12. Given an array-based stack `stack[MAX_SIZE]` and a top index initialized to -1, which C code snippet correctly implements `push(int value)`?
- a) if (`top < MAX_SIZE`) `stack[top++] = value;`
 - b) if (`top < MAX_SIZE - 1`) `stack[++top] = value;`
 - c) if (`top < MAX_SIZE`) `stack[++top] = value;`
 - d) if (`top > 0`) `stack[top--] = value;`
13. Given the same setup (top at -1), which C code snippet correctly implements `pop()` (returning the value)?
- a) if (`top > -1`) `return stack[top--];`
 - b) if (`top > 0`) `return stack[--top];`
 - c) if (`top > -1`) `return stack[--top];`
 - d) if (`top >= 0`) `return stack[top+1];`
14. (Tricky) An array-based stack has `MAX_SIZE = 5` and top is initialized to -1. After `push(5)`, `push(10)`, `push(15)`, what is the value of top?
- a) 1
 - b) 2
 - c) 3
 - d) 15
15. What is the condition to check if an array-based stack (with top at -1 initially) is full?
- a) `top == 0`
 - b) `top == MAX_SIZE`

- c) `top == MAX_SIZE - 1`
- d) `top == -1`

16. (Tricky) A stack is implemented in an array `A[0...N-1]`. The top pointer is initialized to N. The push operation is `A[--top] = value` and pop is `value = A[top++]`. What is the condition for `isEmpty()`?
- a) `top == 0`
 - b) `top == -1`
 - c) `top == N`
 - d) `top == N - 1`
17. What is the average-case time complexity of the push operation in a standard (non-resizing) array-based implementation?
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n^2)$
18. (Tricky) What is the worst-case time complexity of the push operation in a dynamic array (e.g., Java's ArrayList) implementation, which resizes when full?
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(1)$ amortized, but $O(n)$ worst-case.
19. What is the space complexity of a stack containing n elements?
- a) $O(1)$
 - b) $O(n)$
 - c) $O(\log n)$
 - d) $O(n^2)$
20. (Tricky) What is the time complexity to find the 3rd element from the bottom of a standard stack, assuming you only have `push`, `pop`, `peek`, and `isEmpty`?
- a) $O(1)$
 - b) $O(3)$
 - c) $O(n)$
 - d) $O(\log n)$

21. What is the time complexity to search for a specific element (not at the top) in a stack?

- a) $O(1)$
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n \log n)$

22. (Tricky) An implementation of a stack uses an array and initializes $\text{top} = 0$. The $\text{push}(\text{value})$ operation is $\text{stack}[\text{top}++] = \text{value}$. What must the $\text{pop}()$ operation (returning the value) be?

- a) $\text{return stack}[\text{top}--];$
- b) $\text{return stack}[--\text{top}];$
- c) $\text{return stack}[\text{top}];$
- d) $\text{return stack}[\text{top}+1];$

SECTION 3: APPLICATIONS – EXPRESSIONS

23. Stacks are used for evaluating Postfix expressions (Reverse Polish Notation). What is the result of evaluating the Postfix expression: $5\ 3\ 2\ * + 4 -$?

- a) 7
- b) 9
- c) 5
- d) 3

24. What is the Postfix equivalent of the Infix expression: $A + B * C / D$? (Assume standard operator precedence, $*$ and $/$ have equal precedence and are left-associative.)

- a) $A\ B\ C\ * D / +$
- b) $A\ B + C * D /$
- c) $A\ B\ C\ * + D /$
- d) $+ A * B / C D$

25. (Tricky) What is the Postfix equivalent of the Infix expression: $(A + B) * (C - D)$?

- a) $A\ B + C\ D - *$
- b) $A\ B + * C\ D -$
- c) $A\ B\ C\ D + * -$
- d) $* + A\ B - C\ D$

26. When converting Infix to Postfix using a stack, what action is taken when an operator of

lower precedence is encountered than the operator at the top of the stack?

- a) Push the new operator onto the stack.
- b) Pop the operator from the stack and append it to the Postfix string, then push the new operator.
- c) Pop all operators from the stack and append them to the Postfix string.
- d) Pop the operator from the stack and append it to the Postfix string, then repeat the check with the new stack top.

27. What is the Prefix (Polish Notation)

equivalent of the Infix expression: $A + B$?

- a) $A\ B\ +$
- b) $+ A\ B$
- c) $A + B$
- d) $B\ A\ +$

28. You are evaluating the Postfix expression $8\ 4\ / 2\ *$. What is the state of the stack just before the final $*$ operation?

- a) $[2, 2]$ (Top is 2)
- b) $[8, 4]$ (Top is 4)
- c) $[2]$ (Top is 2)
- d) $[4]$ (Top is 4)

29. (Tricky) What is the result of evaluating the Postfix expression: $10\ 2\ 8\ * + 3 -$?

- a) 23
- b) 19
- c) 77
- d) 25

SECTION 4: APPLICATIONS – BALANCING, REVERSING, & RECURSION

30. Which of the following expressions has balanced parentheses (checked using a stack)?

- a) $\{(a + b) * [c - d]\}$
- b) $\{[()]\}$

- c) {[()]}
- d) ([[]])

31. How does a stack help in checking for balanced parentheses?

- a) Push all characters onto the stack, then pop and check.
- b) Push opening brackets; on finding a closing bracket, pop and check if it matches.
- c) Push opening brackets; on finding a closing bracket, push it too.
- d) Push all opening brackets, then push all closing brackets, then compare.

32. A common application of the function call stack is recursion. What error is caused by a recursive function that never reaches its base case?

- a) Null Pointer Exception
- b) Stack Underflow
- c) Stack Overflow
- d) Array Index Out of Bounds

33. What information is NOT typically stored in a stack frame on the function call stack?

- a) Return address
- b) Local variables
- c) Function parameters
- d) The global heap pointer

34. (Tricky) Analyze the following C pseudocode:

```
void printReverse(char *str) {
    Stack s = createStack();
    for (int i = 0; str[i] != '\0'; i++) {
        push(s, str[i]);
    }
    while (!isEmpty(s)) {
        printf("%c", pop(s));
    }
}
printReverse("stack");
```

What is the output?

- a) stack

- b) kcats

- c) s

- d) k

35. Given a simple recursive function in Java:

```
int factorial(int n) {
    if (n == 0)
        return 1;
    else
        return n * factorial(n - 1);
}
```

When factorial(4) is called, what is the maximum number of factorial stack frames on the call stack at any given time (including the factorial(4) call)?

- a) 3
- b) 4
- c) 5
- d) 1

SECTION 5: TRICKY VARIATIONS & CODE ANALYSIS

36. (Tricky) How can you implement a Queue using two Stacks (S1 and S2) for efficient dequeue (amortized $O(1)$)?

- a) enqueue: push to S1. dequeue: push all from S1 to S2, pop from S2.
- b) enqueue: push to S1. dequeue: pop from S1.
- c) enqueue: push to S1. dequeue: push all from S1 to S2, pop from S2, then push all back to S1.
- d) enqueue: push to S1. dequeue: if S2 is empty, push all from S1 to S2. Pop from S2.

37. Following the standard two-stack Queue implementation (from Q36), what is the amortized time complexity of the dequeue operation?

- a) $O(n)$
- b) $O(\log n)$

- c) $O(1)$
- d) $O(n^2)$

38. (Tricky) How can you implement a Stack using two Queues (Q1 and Q2), such that push is the costly operation ($O(n)$)?

- a) push: enqueue to Q2, move all from Q1 to Q2, swap Q1/Q2. pop: dequeue from Q1.
- b) push: enqueue to Q1. pop: dequeue from Q1.
- c) push: enqueue to Q1. pop: reverse Q1, dequeue, reverse back.
- d) It is impossible.

39. What is the time complexity of the pop operation in the two-queue stack implementation described in the previous question (where push is $O(n)$)?

- a) $O(1)$
- b) $O(n)$
- c) $O(\log n)$
- d) $O(1)$ amortized

40. A "Min Stack" is a stack that supports push, pop, peek, and getMin() all in $O(1)$ time. How is this typically achieved?

- a) By keeping a sorted array of all elements.
- b) By traversing the stack on every getMin() call.
- c) By using a second stack that stores the minimum value seen at or below that element.
- d) It is not possible to achieve $O(1)$ for all operations.

41. (Tricky) You are implementing two stacks in a single array $A[0 \dots N-1]$. Stack 1 starts at index 0 and grows right (top1). Stack 2 starts at index $N-1$ and grows left (top2). What is the condition for "Stack Overflow" (i.e., the array is full)?

- a) $\text{top1} == \text{top2}$
- b) $\text{top1} + 1 == \text{top2}$

- c) $\text{top1} == N-1$ and $\text{top2} == 0$
- d) $\text{top1} == N/2$ and $\text{top2} == N/2 + 1$

42. What is the output of the following Java code?

```
Stack s = new Stack<>();
s.push(10);
s.push(20);
s.push(30);
System.out.print(s.pop());
System.out.print(s.peek());
System.out.print(s.pop());
```

- a) 302020
- b) 302010
- c) 303020
- d) 302030

43. (Tricky) What is the output of the following C pseudocode?

```
Stack s = createStack();
push(s, 5);
push(s, 10);
int x = peek(s);
push(s, x + 5);
pop(s);
pop(s);
printf("%d", peek(s));
```

- a) 5
- b) 10
- c) 15
- d) Error (Stack Underflow)

44. Which data structure would you use to efficiently find the "next greater element" for all elements in an array in $O(n)$ time? (For an element x , find the first element to its right that is greater than x .)

- a) A Queue
- b) A Stack
- c) A Priority Queue
- d) A Hash Map

45. (Tricky) What is the final state of the stack s (shown as bottom...top) after this C pseudocode?

```
Stack s = createStack();
Stack temp = createStack();
push(s, 1);
push(s, 2);
push(s, 3);

// Invert the stack
while (!isEmpty(s)) {
    push(temp, pop(s));
}
while (!isEmpty(temp)) {
    push(s, pop(temp));
}
```

- a) [1, 2, 3] (Top is 3)
- b) [3, 2, 1] (Top is 1)
- c) Empty
- d) [1, 2, 3] (Top is 1)

46. What is the value of x after this Java code snippet?

```
Stack s = new Stack<>();
s.push(5);
s.push(8);
int x = s.pop(); // x = 8
x = x + s.peek(); // x = 8 + 5
s.push(x); // s = [5, 13]
x = s.pop(); // x = 13
```

- a) 5
- b) 8
- c) 13
- d) 18

47. (Tricky) Consider a "Min Stack" that stores pairs (value, currentMin) on a single stack. If we push (5), (6), (3), (4), what is at the top of the stack?

- a) (4)
- b) (4, 3)

- c) (4, 5)
- d) (3)

48. The "Stock Span Problem" finds the number of consecutive days before the current day that had a stock price less than or equal to the current day's price. This problem is efficiently solved using:

- a) A Stack
- b) A Queue
- c) Two Stacks
- d) A Min-Heap

49. (Tricky) Analyze this Java code. What is the final value of sum?

```
Stack s = new Stack<>();
s.push(1);
s.push(2);
s.push(3);
int sum = 0;
while (!s.isEmpty()) {
    sum += s.peek();
    s.pop();
}
```

- a) 1
- b) 3
- c) 6
- d) 0

50. (Tricky) A recursive function to sort a stack s is defined as:

```
void sortStack(Stack s) {
    if (!s.isEmpty()) {
        int temp = s.pop();
        sortStack(s);
        // Missing code: inserts 'temp' in sorted order
        insertSorted(s, temp);
    }
}
```

How many times is sortStack (the recursive function itself) placed on the call stack when sorting a stack with 4 elements? 1 4 5 10

Queue Test Bank (50 Questions)

Category 1 — Conceptual & FIFO Principles

1. **A queue data structure follows which principle?**
 - a) LIFO (Last-In, First-Out)
 - b) FIFO (First-In, First-Out)
 - c) LILO (Last-In, Last-Out)
 - d) FILO (First-In, Last-Out)
2. **Which of the following is a real-world example of a queue?**
 - a) A stack of plates
 - b) The “Undo” feature in a text editor
 - c) A line of people waiting for a bus
 - d) Function calls in a program
3. **Which of the following applications would *not* be suitable for a queue?**
 - a) A print spooler managing print jobs
 - b) A web server handling requests in the order they arrive
 - c) Reversing a string
 - d) A CPU scheduling algorithm (like Round Robin)
4. **The enqueue operation is used to:**
 - a) Remove an item from the front of the queue
 - b) Add an item to the rear of the queue
 - c) Check the item at the front of the queue without removing it
 - d) Check if the queue is empty
5. **The dequeue operation is used to:**
 - a) Remove an item from the front of the queue
 - b) Add an item to the rear of the queue
 - c) Look at the item at the rear of the queue
 - d) Check if the queue is full
6. **What is the key difference between peek (or front) and dequeue?**
 - a) peek adds an item, dequeue removes it
 - b) peek removes an item from the rear,

dequeue removes from the front

c) peek returns the front item without removing it, dequeue removes and returns the front item

d) peek is $O(N)$ complexity, dequeue is $O(1)$

7. **A data structure that allows insertions and deletions from both the front and the rear is called a:**
 - a) Stack
 - b) Priority Queue
 - c) Deque (Double-Ended Queue)
 - d) Binary Tree

Category 2 — Simple Linear Array Implementation (Tricks & Pitfalls)

8. **In a simple linear array queue with front = 0 and rear = -1, which condition checks if the queue is empty?**
 - a) front == rear
 - b) front > rear
 - c) rear == -1
 - d) front == capacity
9. **Queue size 5, initial front=0, rear=-1. After: enqueue(10), enqueue(20), dequeue()**
 - a) front = 0, rear = 0
 - b) front = 1, rear = 1
 - c) front = 1, rear = 0
 - d) front = 0, rear = 1
10. **Primary disadvantage of a simple linear array queue (where dequeue increments front):**
 - a) enqueue operations are $O(N)$
 - b) It can only store a limited number of elements
 - c) The queue “drifts” down the array, leaving unusable space at the beginning
 - d) peek operations are not possible
11. **In a linear array queue with capacity = 10, front = 0, rear = -1, isFull() is:**

- a) rear == front
- b) rear == capacity
- c) rear == capacity - 1
- d) front == -1

12. Time complexity of the following *inefficient* dequeue (to prevent drifting):

```
public int dequeue() {
    int item = arr[0];
    for (int i = 0; i < rear; i++) {
        arr[i] = arr[i + 1];
    }
    rear--;
    return item;
}
```

- a) O(1)
- b) O(N)
- c) O(log N)
- d) O(N²)

13. Linear array queue of size 5 (front=0, rear=-1):

Operations: enqueue(A), enqueue(B), enqueue(C), dequeue(), dequeue(), enqueue(D)

What element is at arr[2]?

- a) B
- b) C
- c) D
- d) (empty)

14. After the operations in Q13, what are front and rear?

- a) front = 2, rear = 2
- b) front = 0, rear = 2
- c) front = 2, rear = 3
- d) front = 1, rear = 2

15. In a non-circular linear queue with array size 10, if front = 5 and rear = 5 (both point

to elements), the logical size is:

- a) 0
- b) 1
- c) 10
- d) 5

Category 3 — Circular Array Implementation (Core & Tricks)

16. Primary problem solved by a circular array queue:

- a) Makes enqueue O(1)
- b) Makes dequeue O(1)
- c) Prevents the queue from ever becoming full
- d) Reuses empty space created by dequeue operations efficiently

17. Circular queue capacity = 5, front = 3, rear = 1. Indices from front to rear are:

- a) 3, 4, 0, 1
- b) 1, 2, 3
- c) 3, 2, 1
- d) 1, 0, 4, 3

18. In this C enqueue for a circular queue, fill the blank:

```
void enqueue(Queue* q, int value) {
    if (isFull(q)) return;
    q->rear = (_____) % q->capacity;
    q->arr[q->rear] = value;
    q->size++;
}
```

- a) `q->rear`
- b) `q->rear + 1`
- c) `q->rear - 1`
- d) `q->front + 1`

19. In this dequeue pseudocode, fill the blank:

```
function dequeue():
```

```
    if isEmpty(): return ERROR
```

```
    item = arr[front]
```

```
    front = (_____) % capacity
```

```
    size--
```

```
    return item
```

- a) `front`
- b) `rear`
- c) `front + 1`
- d) `front - 1`

20. Using “waste one slot” with capacity = 10, front = 0. Which rear means full?

- a) 0
- b) 9
- c) 8
- d) 1

21. With “waste one slot” (capacity = N), isFull() is:

- a) front == rear
- b) size == capacity
- c) (rear + 1) % capacity == front
- d) rear == capacity - 1

22. With “waste one slot”, isEmpty() is:

- a) front == rear
- b) (rear + 1) % capacity == front
- c) front == -1
- d) size == 0

23. Circular queue capacity = 6, “waste one slot”. If front = 4 and rear = 3, the queue is:

- a) Empty
- b) Full
- c) Contains 1 element
- d) Contains 5 elements

24. Circular queue capacity = 6, “waste one slot”. If front = 4 and rear = 4, the queue is:

- a) Empty
- b) Full
- c) Contains 1 element
- d) Invalid state

25. Circular queue capacity = 8 with a size variable. front = 6, rear = 2, size = 4. The next element dequeued is at index:

- a) 6
- b) 2
- c) 3
- d) 7

26. Using Q25’s state, after enqueue(50), front and rear become:

- a) front = 6, rear = 3
- b) front = 7, rear = 2
- c) front = 6, rear = 2 (operation fails)
- d) front = 7, rear = 3

27. Using Q25’s state, after dequeue(), front and rear become:

- a) front = 6, rear = 2
- b) front = 7, rear = 2
- c) front = 5, rear = 2
- d) front = 6, rear = 1

28. Circular queue size 5 (indices 0–4), “waste one slot” (front=0, rear=0 when empty).

Trace:

enqueue(A), enqueue(B), enqueue(C),
dequeue(), enqueue(D), enqueue(E),
dequeue()

What is at the front now?

- a) B
- b) C
- c) D
- d) E

29. After the operations in Q28, final front and rear are:

- a) front = 2, rear = 4
- b) front = 2, rear = 0

- c) front = 1, rear = 4
- d) front = 0, rear = 2

30. Bug in this circular queue enqueue (waste one slot):

```
public void enqueue(int item) {
    if ((rear + 1) % capacity == front) {
        System.out.println("Queue is full");
    } else {
        arr[rear] = item;
        rear = (rear + 1) % capacity; // Line X
    }
}
```

- a) The `isFull` check is wrong
- b) `rear` is updated after inserting, so it inserts at the wrong spot
- c) `rear` should be updated before inserting
- d) It doesn't handle the `isEmpty` case

31. Circular queue capacity = 5, front = 3, rear = 1, size = 3. Elements: [_, 20, _, 10, 30] (10 is at front, 20 at rear). After enqueue(40), array is:**

- a) [40, 20, _, 10, 30]
- b) [_, 20, 40, 10, 30]
- c) [_, 20, _, 10, 40]
- d) [_, 40, _, 10, 30]

32. With front, rear, and size, the correct formula to locate the rear slot on enqueue is:

- a) rear = (rear + 1) % capacity
- b) rear = (front + size) % capacity
- c) rear = (rear + size) % capacity
- d) rear = (front - 1) % capacity

33. Primary benefit of % in a circular queue:

- a) Calculates the size
- b) Wraps front/rear to the beginning of the array

- c) Checks if the queue is full
- d) Halves space complexity

34. Circular queue starts with front = -1, rear = -1. Correct handling of the first enqueue:

- a) arr[0] = item; front = 0; rear = 0;
- b) arr[0] = item; rear = 0;
- c) front = 0; rear = (rear + 1) % capacity; arr[rear] = item;
- d) arr[rear] = item; rear++;

35. Circular queue ("waste one slot") with capacity = 10, front = 8, rear = 2. How many elements?

- a) 4
- b) 5
- c) 6
- d) 3

36. Using Q35, the physical index of the 3rd element (1st is at front) is:

- a) 0
- b) 1
- c) 10
- d) 9

Category 4 — Time Complexity

37. Time complexity of enqueue in a circular array queue:

- a) O(1)
- b) O(N)
- c) O(log N)
- d) O(N) worst case, O(1) average

38. Time complexity of dequeue in a circular array queue:

- a) O(1)
- b) O(N)
- c) O(log N)
- d) O(N) worst case, O(1) average

39. Time complexity of peek / front:

- a) O(1)

- b) $O(N)$
- c) $O(\log N)$
- d) $O(0)$

40. Time complexity to find the *maximum* element in an array-based queue (no special ordering):

- a) $O(1)$
- b) $O(\log N)$
- c) $O(N)$
- d) $O(N \log N)$

41. Time complexity of `isFull()` in a circular array queue:

- a) $O(1)$
- b) $O(N)$
- c) $O(\log N)$
- d) $O(\text{capacity})$

42. Worst-case space complexity of an array-based queue with fixed capacity N :

- a) $O(1)$
- b) $O(\log N)$
- c) $O(N)$
- d) $O(N^2)$

Category 5 — Applications & Advanced Scenarios

43. Which graph traversal inherently uses a queue?

- a) Depth-First Search (DFS)
- b) Breadth-First Search (BFS)
- c) Dijkstra's Algorithm
- d) Prim's Algorithm

44. Java (circular queue, capacity 5, "waste one slot"). Initial: `front = 0, rear = 0` (empty). Then:

- `enqueue(1); enqueue(2); enqueue(3); enqueue(4); enqueue(5);`
- a) Queue contains {1, 2, 3, 4, 5}
- b) Queue contains {1, 2, 3, 4}
- c) Throws "Queue is full" on the 5th enqueue

d) Queue contains {1, 2, 3, 4} and rear points to index 4

45. Given the state after Q44 (`front=0, rear=4`, elements {1,2,3,4}), after:

`dequeue(); enqueue(5);`

- a) `front=1, rear=0`, queue: {2, 3, 4, 5}
- b) `front=1, rear=4`, queue: {2, 3, 4, 5}
- c) `front=0, rear=0`, queue: {5, 2, 3, 4}
- d) `front=1, rear=0`, queue: {5, 2, 3, 4}

46. "Hot potato" game with 7 people; pass 10 times; who's eliminated? The simulation can be modeled efficiently using a:

- a) Stack
- b) Queue
- c) Hash Table
- d) Binary Search Tree

47. You need to reverse the order of elements in a queue. Ideal combination:

- a) One queue and one stack
- b) Two stacks
- c) Two queues
- d) An array and two pointers

48. Consider the pseudocode:

`create_queue(Q)`

`create_stack(S)`

`enqueue(Q, 1)`

`enqueue(Q, 2)`

`enqueue(Q, 3)`

`push(S, dequeue(Q))`

`push(S, dequeue(Q))`

`enqueue(Q, pop(S))`

`enqueue(Q, 4)`

`push(S, dequeue(Q))`

What ends up at the front of Q?

- a) 1

- b) 2
- c) 3
- d) 4

49. **Linear (non-circular) array queue size 4,**

front=0, rear=-1:

Operations: enqueue(A), enqueue(B),
dequeue(), dequeue(), enqueue(C),
enqueue(D)

Final state is:

- a) Queue: {C, D}, front=2, rear=3
- b) Queue: {A, B}, front=0, rear=1
- c) Overflow error
- d) Queue: {C, D}, front=0, rear=1

50. **Why prefer $\text{rear} = (\text{rear} + 1) \% \text{capacity}$ over:**

if (rear == capacity - 1) rear = 0; else rear++;

- a) The if statement is $O(N)$
- b) The modulo operator is computationally faster
- c) The if statement fails if rear is not capacity - 1
- d) % handles wrap-around in one clean expression

.30 MCQs — Linked Lists (Concepts & Complexity)

1. Which operation is $\Theta(1)$ on a singly linked list when a pointer to the head is available (no tail pointer)?
 - A. Append at end
 - B. Prepend at head
 - C. Access k-th element
 - D. Delete last element
2. In a singly linked list without extra metadata, computing the length n takes:
 - A. $\Theta(1)$ time, $\Theta(1)$ space
 - B. $\Theta(n)$ time, $\Theta(1)$ space
 - C. $\Theta(\log n)$ time, $\Theta(1)$ space
 - D. $\Theta(n)$ time, $\Theta(n)$ space
3. Compared to dynamic arrays, a primary *disadvantage* of singly linked lists is:
 - A. Expensive insert at head
 - B. Poor cache locality and no $O(1)$ random access
 - C. Must allocate one large contiguous block
 - D. Higher risk of overflow
4. In a circular singly linked list with only a tail pointer, inserting at the “front” (head position) is:
 - A. $\Theta(n)$
 - B. $\Theta(1)$
 - C. $\Theta(\log n)$
 - D. $\Theta(n \log n)$
5. Which is generally true for deleting a node **when only a pointer/reference to that node** in a singly linked list is given (no head pointer)?
 - A. Always possible in $\Theta(1)$
 - B. Impossible unless list is doubly linked
 - C. Possible in $\Theta(1)$ except for the tail node
 - D. Possible but $\Theta(n)$
6. What’s the typical time complexity to find the middle node using slow/fast pointers in a singly linked list of size n ?
 - A. $\Theta(1)$
 - B. $\Theta(\log n)$
 - C. $\Theta(n)$
 - D. $\Theta(n \log n)$
7. Floyd’s cycle-finding algorithm (tortoise-hare) detects a cycle in:
 - A. $\Theta(\log n)$ time, $\Theta(1)$ space
 - B. $\Theta(n)$ time, $\Theta(1)$ space
 - C. $\Theta(n)$ time, $\Theta(n)$ space
 - D. $\Theta(n \log n)$ time, $\Theta(1)$ space
8. Merging two already-sorted singly linked lists of total length n (by relinking nodes) takes:
 - A. $\Theta(n)$ time, $\Theta(1)$ extra space
 - B. $\Theta(n \log n)$ time, $\Theta(1)$ extra space
 - C. $\Theta(n)$ time, $\Theta(n)$ extra space
 - D. $\Theta(n \log n)$ time, $\Theta(n)$ extra space
9. For a singly linked list without a tail pointer, the time to append one element at the end is:
 - A. Amortized $\Theta(1)$
 - B. Worst-case $\Theta(1)$
 - C. $\Theta(n)$
 - D. $\Theta(\log n)$
10. A doubly linked list improves which operation over a singly linked list?
 - A. Random access
 - B. Delete a known node (given pointer) without head traversal
 - C. Search by value
 - D. Space overhead
11. A sentinel (dummy) head node primarily helps to:
 - A. Reduce time complexity of search
 - B. Eliminate null checks at boundaries and simplify uniform insert/delete
 - C. Provide $O(1)$ random access
 - D. Decrease memory overhead per node
12. For a circular linked list, the usual termination condition for traversal is:

- A. `node == nullptr`
 - B. `node->next == nullptr`
 - C. Return to the starting node
 - D. Reaching a sentinel with `key = -∞`
13. A key difference between linked lists and arrays regarding memory is that linked lists:
- A. Require contiguous memory
 - B. Are typically better for CPU caches
 - C. Allow non-contiguous allocation, reducing large-block allocation pressure
 - D. Cannot be resized
14. Sorting a singly linked list is commonly done with mergesort because:
- A. Mergesort doesn't need extra memory on lists
 - B. Mergesort doesn't require random access and is stable
 - C. Quicksort is $\Theta(n)$ on lists
 - D. Heapsort is not in-place on lists
15. Reversing a singly linked list in place typically has time/space:
- A. $\Theta(n)$ time, $\Theta(1)$ extra space
 - B. $\Theta(\log n)$ time, $\Theta(\log n)$ space
 - C. $\Theta(n)$ time, $\Theta(n)$ space
 - D. $\Theta(n \log n)$ time, $\Theta(1)$ space
16. In an LRU cache backed by a doubly linked list + hash map, the list's role is primarily to:
- A. Provide $O(1)$ search by key
 - B. Maintain usage order for $O(1)$ eviction/move-to-front
 - C. Provide $O(1)$ random access
 - D. Compress memory usage
17. If you store a size field (node count) in a linked list and maintain it on updates, the time to get length becomes:
- A. $\Theta(1)$ but updates remain $\Theta(1)$
 - B. $\Theta(1)$ but updates may incur extra bookkeeping
 - C. $\Theta(\log n)$
 - D. Still $\Theta(n)$
18. Removing duplicates from an **unsorted** singly linked list using hashing typically costs:
- A. $\Theta(n)$ time, $\Theta(1)$ extra space
 - B. $\Theta(n \log n)$ time, $\Theta(1)$ extra space
 - C. $\Theta(n)$ time, $\Theta(n)$ extra space
 - D. $\Theta(n^2)$ time, $\Theta(1)$ extra space
19. In a singly linked list, to split into two halves efficiently you would typically:
- A. Use recursion and count depth
 - B. Use slow/fast pointers; slow marks the split point
 - C. Use a stack of nodes
 - D. Require a tail pointer
20. A circular queue can be implemented cleanly using:
- A. Singly linked list with only a head pointer
 - B. Singly linked list with a tail pointer (`tail->next = head`)
 - C. Doubly linked list without tail
 - D. Array only
21. Which statement about iterator invalidation is generally true for linked lists (vs dynamic arrays)?
- A. Insertions/deletions don't invalidate *other* node references/iterators
 - B. Any insertion invalidates all references
 - C. Any deletion invalidates all references
 - D. Reallocations are frequent and invalidate references
22. In a doubly linked list with sentinels at both ends (head/tail dummies), removing the last real element is:
- A. Special-cased and error-prone
 - B. Same code path as any middle deletion (uniform), $\Theta(1)$
 - C. Requires scanning from head
 - D. Requires scanning from tail
23. Detecting whether two singly linked lists intersect (share a physical node) can be done

in $\Theta(n) + \Theta(m)$ time by:

- A. Comparing all pairs of nodes
- B. Hashing values
- C. Checking if their tail node pointer is identical, then aligning lengths and scanning
- D. Sorting both lists then scanning

24. For skip lists (a layered linked list structure), the expected time for search is:

- A. $\Theta(1)$
- B. $\Theta(\log n)$
- C. $\Theta(\sqrt{n})$
- D. $\Theta(n)$

25. An XOR linked list stores, for each node:

- A. Two absolute pointers (prev, next)
- B. One integer index
- C. XOR of prev and next addresses to reduce pointer storage
- D. A hash of neighbors

26. The main drawback of XOR linked lists in modern environments is:

- A. Poor asymptotic complexity
- B. Hard to debug, unsafe with managed memory/GC, and incompatible with many language runtimes
- C. Requires contiguous memory
- D. Requires recursion

27. When concatenating two singly linked lists in $\Theta(1)$ time, you typically need:

- A. Only head pointers
- B. Head and tail pointers for both lists
- C. Only tail pointers
- D. Size fields

28. To check if a singly linked list is a palindrome in $O(n)$ time and $O(1)$ extra space (ignoring recursion), you generally:

- A. Use a queue
- B. Reverse the first half in place, compare halves, then restore
- C. Sort and compare
- D. Use hashing of values

29. Suppose you must delete the k -th node (1-indexed) from the end in a singly linked list in one pass. A common approach is to:

- A. Use two pointers offset by k nodes, then advance together
- B. Use a stack of all nodes
- C. Count length, then traverse again
- D. Use binary search

30. Compared to arrays, linked lists are usually preferable when:

- A. Frequent random indexing is required
 - B. Frequent insertions/deletions near known positions (with pointers) dominate and memory is fragmented
 - C. Sorting speed is paramount
 - D. Data must be in contiguous cache-friendly blocks
-

Notation Conversion Test Bank (Extended)

1. [Easy] What is the primary characteristic of Prefix notation (also known as Polish Notation)?

- A) Operators are placed between their operands.
- B) Operators are placed after their operands.
- C) Operators are placed before their operands.
- D) All operands must be in parentheses.

2. [Easy] Convert the infix expression $A * B + C$ to postfix notation.

- A) $A B C + *$
- B) $A B * C +$
- C) $* A B + C$
- D) $A * B C +$

3. [Easy] Convert the infix expression $A + B / C$ to prefix notation.

- A) $+ A / B C$
- B) $/ + A B C$
- C) $A B C / +$
- D) $A B / C +$

4. [Medium] What is the infix equivalent of the postfix expression $A B C + * D -$?

- A) $A * (B + C) - D$
- B) $(A * (B + C)) - D$
- C) $A * B + C - D$
- D) $A - D * (B + C)$

5. [Medium] What is the infix equivalent of the prefix expression $- / A B * C D$?

- A) $A / B - C * D$
- B) $(A / B) - (C * D)$
- C) $(A / (B - C)) * D$
- D) $A / (B - C * D)$

6. [Hard] Convert the infix expression $(A + B) * (C - (D / E))$ to postfix.

- A) $A B + C D E / - *$
- B) $A B + C * D E / -$
- C) $A B + C D E - / *$
- D) $* + A B - C / D E$

7. [Hard] The operator $^$ (exponentiation) is right-associative. What is the postfix conversion of $A ^$

$B ^ C$?

- A) $A B ^ C ^$
- B) $A B C ^ ^$
- C) $^ ^ A B C$
- D) $A B C ^ ^$

8. [Hard] What is the prefix equivalent of the postfix expression $A B - C D + *$?

- A) $* - A B + C D$
- B) $* A - B + C D$
- C) $A B C D - + *$
- D) $- * A B + C D$

9. [Hard] What is the postfix equivalent of the prefix expression $+ * A B - C D$?

- A) $A B * C D - +$
- B) $A B C D * - +$
- C) $A B * + C D -$
- D) $* A B - C D +$

10. [Medium] When converting infix to postfix using a stack, if the incoming operator has the **same precedence** as the operator on top of the stack (e.g., incoming $*$ and stack top $/$), what action is taken for **left-associative** operators?

- A) Push the incoming operator onto the stack.
- B) Pop the operator from the stack and append it to the output, then push the incoming operator.
- C) Discard the incoming operator.
- D) Append the incoming operator directly to the output, bypassing the stack.

11. [Medium] Convert the infix expression $A - B - C$ to postfix, assuming $-$ is left-associative.

- A) $A B - C -$
- B) $A B C - -$
- C) $A B - - C$
- D) $A - B C -$

12. [Medium] Convert the infix expression $A - (B - C)$ to prefix.

- A) $- - A B C$
- B) $- A - B C$
- C) $- A B - C$
- D) $- A (- B C)$

13. [Medium] Given operator precedence $^ > * = / > + = -$ and standard associativity, the postfix of $A + B * C - D$ is:

- A) $A B C * + D -$
- B) $A B + C * D -$
- C) $A B * C + D -$
- D) $A B C + * D -$

14. [Hard] Convert the infix expression $(A \wedge B) \wedge C$ to prefix, given $\wedge$ is right-associative.

- A) $\wedge \wedge A B C$
- B) $\wedge A \wedge B C$
- C) $\wedge A B \wedge C$
- D) $\wedge B A C$

15. [Hard] In infix-to-postfix conversion, how should **unary minus** be treated to avoid ambiguity?

- A) Treat it as a lower-precedence operator than binary minus.
- B) Treat it as a distinct operator with higher precedence than $\wedge$.
- C) Treat it as the same as binary minus.
- D) Ignore unary minus during conversion.

16. [Medium] The postfix for the infix $-A + B$ (where the minus before A is **unary**) is:

- A) $A - B +$
- B) $- A B +$
- C) $A \text{ neg } B +$ (interpret unary as neg)
- D) $A B + -$

17. [Medium] Convert $(A + B) / (C - D * E)$ to prefix.

- A) $/ + A B - C * D E$
- B) $/ + A B - * C D E$
- C) $/ A B + - C D * E$
- D) $- / + A B C * D E$

18. [Medium] Which statement about **parentheses** in conversion is true?

- A) Parentheses only affect output, never the operator stack.
- B) A right parenthesis triggers popping operators until the matching left parenthesis.

C) A left parenthesis is immediately output.

D) Parentheses can be ignored if precedence is known.

19. [Medium] If operands can be **multi-digit** (e.g., A12, B3), a robust conversion algorithm should:

- A) Split operands into single characters for uniformity.
- B) Require separators between every character.
- C) Tokenize operands as complete units before applying the stack rules.
- D) Replace multi-digit operands with temporary letters first.

20. [Hard] Consider **function calls** with highest precedence and arguments separated by commas (e.g., $f(A, B + C)$). The prefix of $g(A, f(B, C + D))$ is:

- A) $g A f B + C D$
- B) $g (A , f (B , + C D))$
- C) $g A f B C D +$
- D) $g f A B + C D$